IEEE 830 STANDARD FORMAT

Software Requirements Specification (SRS)

DAFCustomer Support and Monitoring System • Version 1.0 • DAF Tech Engineering

1. Introduction & Product Overview

The DAFCustomer Support and Monitoring System is a dedicated enterprise web application engineered to organize proactive client engagement, record qualitative feedback, and measure support personnel effectiveness. The system decouples client accounts into a strict 3-tier structure (**Organization > Project > Project Module**) to support enterprise contracts where client organizations maintain multiple independent systems and specialized modules.

2. System Actors & Role-Based Access Control (RBAC)

Actor / Role	System Responsibilities	Permissions Assigned
Support Agent	Accesses daily contact queue; conducts outreach; logs structured customer feedback; tracks individual daily/weekly contact completion targets.	`Schedule.ViewOwn`, `Feedback.Create`, `Feedback.ViewOwn`, `Performance.ViewOwn`
Team Lead	Oversees agent workload allocation; generates weekly schedules; tracks overdue contacts; monitors team completion rate and blocker escalations.	All Agent permissions + `Schedule.Generate`, `Schedule.Publish`, `Assignment.Manage`, `Performance.ViewTeam`
Director	Monitors overall customer satisfaction (CSAT) trends, unresolved critical blockers, module health heatmaps, and enterprise-wide staff KPI rankings.	`Dashboard.ViewExecutive`, `Performance.ViewAll`, `Report.ExportAll` (Read-Only)
System Administrator	Manages master entities (Organizations, Projects, Modules, Contacts), creates users, and configures dynamic roles and security privileges.	Full System CRUD: `User.Manage`, `Role.Manage`, `MasterData.Manage`

3. Specific Functional Requirements

- **FR-01 (Hierarchy Decoupling):** The system must independently manage Organizations, Projects under an Organization, and Project Modules under a Project. Contact Persons can link at the Organization, Project, or Module level.
- **FR-02 (Contact Resolution Fallback):** In schedule generation, if a specific Module contact is absent, the system must cascade upwards: Module Contact → Project Contact → Organization Primary Contact.
- **FR-03 (Schedule Generation & State):** The system generates weekly schedules (Monday–Sunday) based on active assignments. Schedules transition from `Draft` to `Published`. Only items in `Published` status count towards performance denominator calculations.
- **FR-04 (Feedback Capture):** Feedback submission requires: Rating (1–5), Priority (Low, Medium, High, Blocker), Status (Not Resolved, Assigned, On Progress, Resolved), and Remarks. Caller Identity (`CallerUserId`) must be extracted automatically from the authenticated JWT token.
- **FR-05 (Evaluation Math):** Daily/Weekly Agent Performance % must be calculated as: `Performance (%) = (Completed Scheduled Contacts / Total Planned Contacts) × 100`.

4. REST API Endpoint Specifications

Method	Endpoint Path	Required Role / Claim	Description
POST	`/api/v1/auth/login`	Public (Anonymous)	Authenticates user; returns JWT token + Refresh Token.
GET	`/api/v1/organizations/{id}/projects`	`Organization.View`	Retrieves projects associated with a customer organization.
POST	`/api/v1/projects/{id}/modules`	`MasterData.Manage`	Attaches a master module to a specific customer project.
GET	`/api/v1/contacts`	`Contact.View`	Filters contacts by OrgId, ProjectId, or ProjectModuleId.
POST	`/api/v1/schedules/generate`	`Schedule.Generate`	Generates batch schedule items for upcoming Monday–Sunday cycle.
PUT	`/api/v1/schedules/{id}/publish`	`Schedule.Publish`	Locks and publishes schedule; activates agent queues.

Method	Endpoint Path	Required Role / Claim	Description
GET	`/api/v1/schedules/my-queue`	`Schedule.ViewOwn`	Returns today's planned contact list for logged-in support agent.
POST	`/api/v1/feedbacks`	`Feedback.Create`	Saves feedback, sets schedule item to Completed, stamps caller.
GET	`/api/v1/performance/summary`	`Performance.ViewOwn`	Returns daily and weekly completion rates and CSAT averages.

5. Implementation Architecture & Delivery Standards

> **Response Envelope Convention:** All backend endpoints must return a standardized wrapper:
> `{ "success": true, "data": { ... }, "message": "Success", "errors": [] }`

- **Backend (.NET 9):** Implemented using Clean Architecture. MediatR / CQRS pattern for command-query separation. FluentValidation handles domain rules (e.g., Rating between 1 and 5). Use EF Core 9 compiled queries for dashboard performance.
- **Frontend (Angular 18+):** Reactive Forms with strict typing. Route guards enforce JWT and granular permission checks. Use an HTTP Interceptor for automatic Bearer token injection and seamless 401 refresh token retry.
- **Security & Audit:** Passwords hashed using PBKDF2/Argon2 via ASP.NET Core Identity. Soft deletes (`IsActive`) enforced on master entities to prevent broken audit linkages.